OPEN SOURCE SOFTWARE — PROJECT REPORT

VITyarthi | NGMC

The Open Source Audit

Case Study on LibreOffice

Course: Open Source Software

Student Name: ADNAN KHAN

Registration Number: 24MEI10121

Table of Contents

Introduction 3
Part A1 — Origin and Problem Statement 4
Part A2 — Licence Analysis 5
Part A3 — Ethics of Open Source 7
Part B — Linux Footprint 9
Part C — FOSS Ecosystem 11
Part D — Open Source vs Proprietary 12
Part E — Shell Scripts 13
Conclusion 14
Final Reflection 15

INTRODUCTION

Introduction

Open source software forms the backbone of modern computing systems. From operating systems to web servers and programming languages, a substantial portion of today's digital infrastructure is built on software that is freely available, modifiable, and distributable under licences that protect the rights of both developers and end users.

This report presents a structured audit of LibreOffice — one of the most widely deployed open-source office suites in the world. The audit spans six analytical dimensions: its origin and governance history, its licensing model, the ethics underpinning the open source movement, its presence on Linux systems, its position within the broader FOSS ecosystem, and a comparative evaluation against its primary proprietary competitor, Microsoft Office.

The purpose of this project is not simply to catalogue facts about LibreOffice. It is to understand the deeper principles that made LibreOffice necessary, the decisions that shaped its development, and what its story reveals about the relationship between software freedom, corporate power, and community governance. Additionally, the project includes practical shell scripting exercises to demonstrate hands-on engagement with Linux-based open source environments.

Why LibreOffice?

LibreOffice is a particularly instructive case study because it did not emerge from nothing. It emerged from a crisis — a breakdown of trust between a developer community and a corporate owner. Understanding that crisis illuminates the principles that make open source meaningful beyond mere licensing.

PART A1

Origin and Problem Statement

1.1 The Need for a Free Office Suite

In the late 1990s, desktop productivity software was almost entirely controlled by Microsoft. Microsoft Office was expensive, ran primarily on Windows, and used proprietary file formats that effectively locked users into the Microsoft ecosystem. For schools, governments, and individuals in lower-income regions, this created a fundamental access problem: either pay substantial licensing fees or be excluded from the standard tools of modern work.

The situation changed significantly in 1999 when Sun Microsystems acquired a German company called Star Division, which had developed a competing office suite called StarOffice. Sun made a strategic decision to release the StarOffice codebase as open source software under the name OpenOffice.org. For the first time, there existed a credible, fully-featured, and free alternative to Microsoft Office that could read and write standard document formats, run on multiple operating systems, and be freely distributed to anyone who needed it.

1.2 The Governance Crisis

OpenOffice.org grew rapidly, accumulating tens of millions of users and a large global community of volunteer contributors. However, beneath this visible success, a structural problem was developing. Sun Microsystems maintained firm control over the project: contributors were required to sign copyright assignment agreements granting Sun joint ownership of their code, and Sun employees held authority over release schedules and project direction. The community built the software; the corporation held the power.

This arrangement became acutely dangerous in January 2010, when Oracle Corporation completed its acquisition of Sun Microsystems for approximately 7.4 billion US dollars. Unlike Sun, which had a genuine — if imperfect — commitment to the open source community, Oracle was primarily known as an aggressive enterprise software company with a history of using intellectual property law competitively. Oracle's lawsuit against Google over Java in Android, filed the same year, confirmed the open source community's worst fears about Oracle's intentions.

Contributors who had assigned copyright to Sun now found that those rights belonged to Oracle. Oracle launched a commercial product called Oracle Open Office built directly on their volunteer labour, and communication between Oracle and the community became sparse and dismissive. The implicit contract of the open source contribution model had been violated.

1.3 The Fork: Birth of LibreOffice

In September 2010, a group of leading OpenOffice.org contributors responded by forming The Document Foundation and launching LibreOffice — a fork of the OpenOffice.org codebase designed to be governed on entirely different principles. The founding manifesto was explicit: no single company would control the project, no copyright assignment would be required, and decisions would be made transparently by elected committees representing the community.

The response was remarkable. Within weeks, hundreds of developers migrated from OpenOffice.org to LibreOffice. Major Linux distributions including Ubuntu, Fedora, and Debian announced they would ship LibreOffice in their next releases. Oracle, offered the chance to donate the OpenOffice.org brand and join the new foundation, declined. By 2011, Oracle had donated the OpenOffice.org codebase to the Apache Software Foundation, effectively conceding that it had lost the community entirely.

Core Problem Statement

LibreOffice was created to solve two overlapping problems: the need for a high-quality, freely available office suite accessible to all users regardless of income, and the need for an open source project governed by its community rather than controlled by a single corporate entity.

PART A2

Licence Analysis

2.1 LibreOffice and the Mozilla Public Licence

LibreOffice is licensed under the Mozilla Public Licence version 2.0 (MPL 2.0), a modern, file-level copyleft licence developed by the Mozilla Foundation. The MPL occupies a middle ground in the spectrum of open source licences: it is stronger than permissive licences such as the MIT or BSD licences, which impose almost no requirements on how derivative works are distributed, but more flexible than the GNU General Public Licence (GPL), which requires the entire derivative work to be released under the same licence.

Under the MPL 2.0, the key obligation is at the file level: any file from the LibreOffice codebase that is modified must be released as open source under the MPL 2.0 when distributed. However, developers may combine MPL-licensed files with files under other licences — including proprietary licences — provided they maintain clear separation. This architecture makes LibreOffice suitable for integration in commercial environments without forcing the entire commercial product to become open source.

2.2 The Four Freedoms

LibreOffice satisfies all four fundamental freedoms of free software as articulated by Richard Stallman and the GNU Project:

1. **Freedom 0 —** The freedom to run the programme for any purpose, without restriction. LibreOffice can be used by individuals, businesses, governments, and educational institutions without licensing fees or usage limitations.
2. **Freedom 1 —** The freedom to study how the programme works and modify it. The complete source code of LibreOffice is publicly available in its repository, and any developer may read, compile, and alter it.
3. **Freedom 2 —** The freedom to redistribute copies so you can help others. LibreOffice may be freely copied and distributed, which is why it is included in virtually all major Linux distributions.
4. **Freedom 3 —** The freedom to distribute copies of your modified versions. Any developer who modifies LibreOffice may release their version, provided MPL-licensed files retain their open source status.

2.3 MPL 2.0 Compared to Other Licences

Understanding why The Document Foundation chose the MPL requires understanding the trade-offs between the major open source licence families.

Licence	Copyleft Strength	Commercial Use	Patent Protection
MIT / BSD	None (permissive)	Unrestricted	No explicit terms
Apache 2.0	None (permissive)	Unrestricted	Explicit grant
MPL 2.0	File-level copyleft	Permitted	Explicit grant

Licence	Copyleft Strength	Commercial Use	Patent Protection
LGPL 2.1	Library-level	Permitted with conditions	Explicit grant
GPL v3	Strong (whole work)	Permitted, source required	Explicit grant

The MPL was selected for LibreOffice because it enables community and commercial contributors to participate without forcing all commercial derivative work to become open source — a practical requirement given that companies like Collabora and Red Hat build commercial products on the LibreOffice codebase — while still ensuring that improvements to the core files flow back to the community.

2.4 Free as in Freedom vs Free as in Beer

A common source of confusion is the dual meaning of the word 'free' in the phrase 'free software.' LibreOffice is free in both senses: it carries no cost to download, install, or use, and it provides users with the freedoms described above. However, the open source community considers freedom from restriction — the ability to study, modify, and redistribute — to be far more significant than the absence of cost.

A proprietary tool can be offered at no cost yet still deny users all four freedoms. Conversely, a paid product could theoretically be open source. LibreOffice's significance lies not in its zero price tag but in the fact that individuals, organisations, and governments who use it are not dependent on any single vendor's decisions about how it works, what data it collects, or whether it will continue to exist.

PART A3

Ethics of Open Source

3.1 Open Source as a Philosophical Commitment

Open source software is not merely a distribution model. It is a philosophical position about knowledge, power, and the obligations of those who build tools that others depend upon. At its core, the open source philosophy holds that software — like mathematics or language — is a form of knowledge that benefits society most when it is shared freely rather than enclosed by proprietary control.

This conviction has practical consequences. When source code is available for inspection, security vulnerabilities can be found and fixed by anyone, not only by the original vendor. When software can be modified, communities can adapt it to their specific cultural, linguistic, or institutional needs. When it can be redistributed without charge, organisations in resource-constrained environments can access the same tools as well-funded institutions.

3.2 The Question of Universal Openness

A natural ethical question follows: should all software be open source? The argument in favour is powerful — open source promotes accountability, enables independent security audits, prevents vendor lock-in, and ensures that public-interest software serves the public rather than shareholders. Governments, schools, and hospitals, which depend on software to deliver essential services, have particularly strong reasons to prefer software whose code they can inspect and whose future they can control.

However, a nuanced view recognises that blanket mandates are unlikely to be practical or even desirable in all cases. Certain types of software — particularly security-critical cryptographic systems or tools with narrow specialised audiences — may carry risks if their internal mechanisms are fully public. Some organisations fund software development by maintaining proprietary advantages; eliminating that model entirely might reduce investment in certain categories of innovation.

The more defensible position is not that all software must be open source, but that software which is funded by public money, used in public institutions, or holds significant power over personal data should be open source by default. The LibreOffice model — open governance, open code, freely distributable — represents the appropriate standard for software of this kind.

> **Ethical Principle**
> When software is built using public funding, used in schools and government offices, or processes citizens' personal data, the public has a legitimate interest in being able to inspect, verify, and modify it. Open source is not a courtesy in these cases — it is an accountability requirement.

3.3 Corporate Participation and the Commons

One of the central ethical tensions in the open source world concerns the relationship between corporate participation and community governance. Companies like Google, Red Hat, IBM, and Microsoft contribute enormous resources to open source projects — code, documentation, infrastructure, and funding. This raises a question: does corporate investment entitle corporations to governance authority?

The LibreOffice story suggests the answer is no, or at least not unconditionally. Sun Microsystems' investment in OpenOffice.org was real and substantial, but it did not give Sun — or its successor Oracle — the right to govern the project in ways that contradicted the values of the community that had built it. When Oracle exercised that authority against the community's interests, the community forked. The fork was costly, but it preserved the principle that the community, not the corporation, is the legitimate steward of a project built by the community.

The LibreOffice governance model codifies this principle structurally. No single company may control more than one-third of The Document Foundation's governing board. Contributors retain copyright on their own code. This is not anti-corporate; several commercial companies actively support LibreOffice. It is pro-community: it ensures that commercial support is welcome while commercial control is structurally impossible.

3.4 Standing on the Shoulders of Giants

Perhaps the most profound ethical dimension of open source is the concept of cumulative, shared knowledge. Every open source project builds on the work of others: programming languages, compilers, operating system kernels, network stacks, and cryptographic libraries that were themselves built by communities of contributors who chose to share rather than enclose.

LibreOffice is a direct beneficiary of this tradition. It builds on decades of contributions to the OpenOffice.org codebase, on Linux, on the C++ standard library, on the work of countless volunteer translators and documentation writers. Its obligation, implicit in the licences it uses and explicit in the values of its founding community, is to extend that tradition — to build something that others can in turn build upon.

This is the ethical core of open source: a recognition that the most significant advances in software are collective achievements, and that treating them as private property misrepresents how they were created and who they ultimately serve.

PART B

Linux Footprint of LibreOffice

4.1 Installation on Linux

LibreOffice is available in the official package repositories of virtually every major Linux distribution. On Debian-based systems such as Ubuntu, it can be installed using the APT package manager with a single command. On Red Hat-based systems such as Fedora or CentOS, the DNF or YUM package manager is used. Arch Linux users may install it from the official repositories using pacman, while users who require the latest version may download a tarball directly from the LibreOffice website.

The installation process is straightforward and requires no manual compilation or complex configuration for standard use. The package manager handles dependency resolution automatically, ensuring that all required libraries are present before installation completes. Typical installation size on a standard Linux system is approximately 300 to 600 megabytes depending on which language packs and extensions are included.

4.2 Filesystem Layout

Once installed via a system package manager, LibreOffice occupies several locations across the Linux filesystem hierarchy:

- **/usr/lib/libreoffice/** — Main installation directory containing the programme binaries, shared libraries, configuration data, and resource files. This directory is managed by the package manager and should not be edited manually.
- **/usr/bin/** — Contains symbolic links (soffice, libreoffice) pointing to the main LibreOffice launcher. Placing links here makes LibreOffice accessible from any directory without specifying the full path.
- **~/.config/libreoffice/** — Per-user configuration directory. Stores user preferences, custom dictionaries, extension data, macro configurations, and UI customisations. This directory is created the first time LibreOffice is launched by a given user.
- **~/.local/share/** — May contain user-installed extensions and templates that are not managed by the system package manager.
- **/var/log/** — System log directory. LibreOffice itself does not typically write to this location, but system-level package manager logs here may record installation and update events.

For enterprise deployments, administrators can deploy system-wide configuration files to /etc/libreoffice/, which take precedence over user-level settings. This allows organisations to standardise defaults for font settings, macro security policies, and default file format choices across all users on a system.

4.3 Process Model and Permissions

LibreOffice runs entirely within the context of the user account that launches it. It does not require root privileges for normal operation and does not run any background daemons or system services. This is consistent with the principle of least privilege: the application can only access files and resources that the current user has permission to access.

When LibreOffice is launched, it spawns a main process (soffice.bin) along with several child processes that handle rendering, font management, and the internal UNO component framework. These processes communicate through interprocess pipes. The quickstarter feature, if enabled, keeps a minimal process resident in memory after the application is closed to reduce subsequent launch times, but this can be disabled by users who prefer a smaller memory footprint.

4.4 Updates and Package Management

On systems where LibreOffice was installed via the distribution's package manager, updates are handled automatically as part of the standard system update process. Running apt upgrade on Ubuntu or dnf upgrade on Fedora will update LibreOffice alongside all other installed software. This integration with the system update mechanism is a significant advantage from a security perspective: known vulnerabilities are patched promptly and without requiring users to separately track software version releases.

Users who install LibreOffice directly from the LibreOffice website — outside the package manager — are responsible for manually checking for and applying updates. The LibreOffice built-in updater can notify users of new releases, but the update process in this case requires downloading and installing a new package manually.

PART C

The FOSS Ecosystem

5.1 LibreOffice Within the Broader FOSS World

LibreOffice does not exist in isolation. It is one component of an interconnected ecosystem of free and open source software that collectively provides a complete, proprietary-free computing environment. Understanding LibreOffice's place in this ecosystem requires understanding both the dependencies it relies upon and the projects it supports or inspires.

5.2 Key Dependencies

LibreOffice's codebase integrates dozens of open source libraries that it depends on for core functionality:

- **ICU (International Components for Unicode)** — Handles Unicode text processing, enabling LibreOffice to work correctly with scripts from Arabic to Chinese to Tamil.
- **HarfBuzz** — An open source text shaping engine that converts Unicode text into properly shaped and positioned glyphs, critical for correct rendering of complex scripts.
- **Cairo and Skia** — Graphics rendering libraries used for drawing document content on screen and for PDF export.
- **Boost C++ Libraries** — A comprehensive set of C++ utilities used throughout the LibreOffice codebase for tasks ranging from string handling to filesystem operations.
- **Python** — LibreOffice includes a full Python runtime, enabling users to write macros and extensions in Python as well as the built-in LibreOffice Basic language.

5.3 The Open Document Format

One of LibreOffice's most significant contributions to the FOSS ecosystem is its robust implementation of and advocacy for the Open Document Format (ODF). ODF is an international standard (ISO/IEC 26300) for office document file formats. Unlike the proprietary OOXML format used by Microsoft Office, ODF is an open standard developed by a public standards body with input from multiple vendors and independent contributors.

By using ODF as its native format, LibreOffice ensures that documents created today can be opened by any standards-compliant application in the future, without dependence on a specific vendor's ongoing cooperation. This is particularly important for governments and archives, where documents must remain accessible for decades. Several governments — including those of Germany, the Netherlands, and various Indian states — have mandated ODF as the official document format for public administration precisely because it eliminates vendor lock-in.

5.4 Governance: The Document Foundation

The Document Foundation (TDF) is a German non-profit organisation (Stiftung) that was established in 2010 to serve as the independent steward of the LibreOffice project. Its governance structure was designed explicitly to prevent the concentration of control that had made OpenOffice.org vulnerable to Oracle's acquisition of Sun.

TDF's charter includes several structural safeguards. No single company may appoint more than a specified proportion of the Board of Directors. The Board is elected by TDF members, who include both individual contributors and organisational members. Major decisions — changes to the project's direction, licence, or governance rules — require supermajority votes. All board meetings are publicly minuted, and financial statements are publicly available.

Companies including Collabora, Red Hat, CIB, and allotropia have contributed developers and funding to LibreOffice through TDF membership. This model — where commercial entities support the project without controlling it — is widely regarded as a template for sustainable open source governance.

PART D

Open Source vs Proprietary: LibreOffice vs Microsoft Office

A structured comparison between LibreOffice and Microsoft Office illustrates both the strengths and the current limitations of the open source model in the productivity software space.

Feature	LibreOffice	Microsoft Office
Cost	Free of charge	Subscription / one-time licence
Source Code	Fully open (MPL 2.0)	Proprietary / closed
Customisation	High — plugins, macros, APIs	Limited to official add-ins
Governance	Community-driven (TDF)	Corporate (Microsoft)
File Formats	ODF native; reads OOXML	OOXML native; partial ODF
Platform	Windows, macOS, Linux	Windows, macOS (limited Linux)
Transparency	Full audit trail possible	Opaque to end users
Security Model	Community-reviewed code	Vendor patches only

6.1 Analysis

The table above reveals that the advantages of LibreOffice are primarily philosophical and financial: it costs nothing, its code is open to inspection, and its governance ensures that no single company can make decisions that harm its user base. These advantages are decisive for certain categories of user — schools, governments, NGOs, and individuals who need a reliable office suite but cannot or prefer not to pay recurring subscription fees.

Microsoft Office retains meaningful advantages in enterprise integration, particularly in organisations already deeply embedded in the Microsoft 365 ecosystem. Its collaboration features — real-time co-authoring in SharePoint and Teams — are more mature than LibreOffice's equivalent offerings, and its commercial support infrastructure makes it the default choice for organisations that require guaranteed service level agreements.

Format compatibility remains a practical challenge. While LibreOffice can open and edit OOXML files (the .docx, .xlsx, .pptx formats used by Microsoft Office), complex documents with advanced formatting, embedded macros, or proprietary fonts may not render identically. Organisations that frequently exchange documents with Microsoft Office users need to test their specific document types before committing to LibreOffice as a primary tool.

6.2 Conclusion on Comparison

LibreOffice is not a lesser Microsoft Office. It is a different kind of software project — one whose value lies as much in its governance and philosophy as in its feature set. For users who prioritise freedom, cost, transparency, and independence from vendor decisions, LibreOffice is not merely an acceptable alternative. It is the better choice. For organisations

deeply integrated into the Microsoft ecosystem and unwilling to invest in migration, Microsoft Office remains pragmatically superior.

The two products are not competing solely on features. They represent two different answers to the question of what software should be and who should control it.

PART E

Shell Scripts: Linux Interaction with LibreOffice

The following five shell scripts demonstrate practical Linux skills relevant to open source software auditing. Each script is described with its purpose, the key commands it uses, and an explanation of its relevance to the LibreOffice audit.

Script	Purpose	Key Commands Used
1 — System Identity	Displays hostname, OS, kernel, uptime, and current user	`uname -a, whoami, uptime, hostname`
2 — Package Inspector	Checks if LibreOffice is installed; shows version and install path	`dpkg -l, which, apt show, grep`
3 — Disk Auditor	Lists directory sizes under /usr/lib/libreoffice and checks permissions	`du -sh, ls -la, stat, df -h`
4 — Log Analyzer	Scans /var/log for errors and counts keyword occurrences	`grep -c, tail, wc -l, awk`
5 — Manifest Generator	Creates a personalised open-source philosophy statement and writes it to a file	`echo, date, tee, cat, chmod`

Script 1 — System Identity

This script gathers and displays foundational information about the Linux environment in which LibreOffice is running. It reports the hostname, full kernel version, operating system distribution name and version, current logged-in user, system uptime, and available memory. Commands used include uname -a for kernel information, cat /etc/os-release for distribution details, whoami for the current user identity, uptime for system runtime, and free -h for a human-readable memory summary.

Relevance: Before auditing any installed software, an administrator needs to establish the baseline characteristics of the system. This script provides a reproducible, timestamped snapshot of the environment that can be archived alongside the audit findings.

Script 2 — Package Inspector

This script checks whether LibreOffice is installed on the current system and, if so, retrieves its version, installation path, and package size. It uses dpkg -l | grep libreoffice to list installed LibreOffice packages, which libreoffice to locate the executable, and apt show libreoffice to retrieve detailed package metadata including the installed size and dependencies. On RPM-based systems, the equivalent commands use rpm -qa and dnf info.

Relevance: Package inspection is a standard step in open source software audits. It confirms that the correct version is deployed, identifies whether the package was installed via the system package manager (and therefore receives automatic updates) or manually (and therefore requires manual update management), and lists the key binary locations for subsequent analysis.

Script 3 — Disk Auditor

This script analyses the disk usage of the LibreOffice installation directory and reports on the permissions and ownership of key files. It uses du -sh /usr/lib/libreoffice/* to show the sizes of subdirectories within the installation, ls -la /usr/bin/libreoffice to inspect the executable symlink and its permissions, and stat /usr/lib/libreoffice/program/soffice.bin to retrieve detailed metadata including inode number, access permissions, and last modification time.

Relevance: Auditing filesystem permissions is essential for security compliance. Incorrectly set permissions on executable files can be exploited by local privilege escalation attacks. This script provides a quick method to verify that the installation conforms to expected security baselines.

Script 4 — Log Analyzer

This script searches system log files for entries relevant to LibreOffice activity. It uses grep -i libreoffice /var/log/syslog to find all mentions of LibreOffice in the system log, grep -c error /var/log/syslog to count error-level log entries across the full system log, and tail -n 50 /var/log/dpkg.log | grep libreoffice to retrieve the most recent package management activity involving LibreOffice. The results are formatted and written to a timestamped output file using tee.

Relevance: Log analysis is a critical component of operational auditing. By examining log files, an auditor can determine when LibreOffice was last updated, whether any errors have been recorded during installation or operation, and whether any unexpected activity has occurred. The automated keyword search enables rapid analysis of large log files that would be impractical to review manually.

Script 5 — Manifest Generator

This script generates a personalised open source philosophy statement and writes it to a file named oss_manifest.txt in the user's home directory. It uses echo to construct the statement line by line, date to embed a timestamp, and tee to write the output both to the terminal and to the file simultaneously. The script concludes by using chmod 644 to set appropriate read permissions on the output file, and cat to display the final result.

Relevance: Beyond its philosophical content, this script demonstrates several foundational shell scripting skills: variable assignment, command substitution, output redirection, file permission management, and the use of heredocs for multi-line output. It also serves as a practical reminder that open source software is not merely a technical artefact but an expression of values about knowledge, freedom, and collaboration.

CONCLUSION

Conclusion

This audit of LibreOffice has traced the project from its origins in the governance crisis of 2010 through its licensing architecture, philosophical foundations, Linux ecosystem presence, and comparative positioning against proprietary alternatives. Several conclusions emerge clearly from this analysis.

First, LibreOffice demonstrates that open source software can be both technically competitive and institutionally resilient. By building The Document Foundation on governance principles that structurally prevent corporate capture, the LibreOffice community created a project whose future is genuinely community-controlled in a way that OpenOffice.org's never was.

Second, the choice of the Mozilla Public Licence 2.0 reflects a mature understanding of the practical requirements of open source sustainability. The file-level copyleft model enables commercial participation — which funds development — while ensuring that modifications to the core codebase remain open. This balance has attracted significant corporate support without compromising community governance.

Third, the ethics of open source extend well beyond the technical question of which licence to use. They encompass questions of power, accountability, and the obligations of those who build tools that others depend upon. LibreOffice's governance model, its support for open document standards, and its commitment to the four software freedoms represent a coherent answer to the question of what software built in the public interest should look like.

Fourth, the practical Linux skills demonstrated through the shell scripting component of this audit — package inspection, filesystem analysis, log analysis, and permission management — are not incidental to the open source audit process. They are central to it. The ability to inspect, verify, and understand the software one depends on is precisely what distinguishes open source systems from their proprietary alternatives.

Final Assessment

LibreOffice is more than a free office suite. It is a working demonstration of the proposition that software built by a community, governed by that community, and freely available to all can compete with and in many respects surpass software built by the largest corporations in the world.

FINAL REFLECTION

Final Reflection

Before undertaking this project, my understanding of open source was largely defined by what it was not: not proprietary, not paid, not controlled by a single company. It was, in my conception, primarily a category of pricing and accessibility. Free software meant software you could download without paying. Open source meant software whose code you could theoretically look at.

This audit has substantially changed that understanding. Open source is not primarily about price or even code availability. It is about governance — about who has the power to make decisions about software that large numbers of people depend upon, and whether those people have any say in those decisions. The LibreOffice story is, at its heart, a story about this question. The community that built OpenOffice.org discovered that the licence under which they had shared their code could not protect them from a governance structure that excluded them. Their response — forking the project and building new governance structures from the ground up — is not a technical event. It is a political one.

What genuinely surprised me in this study was the copyright assignment clause. The image of thousands of developers contributing code in good faith, believing they were adding to a shared commons, and then discovering that their contributions were legally owned by a corporation that had just sued a technology company for billions of dollars — this is genuinely unjust. Not illegal, but unjust. The legal and the ethical are not the same thing, and this case makes that gap viscerally clear.

The community's decision to fork — to walk away from years of accumulated infrastructure and rebuild from scratch rather than accept governance they could not trust — required a kind of principled conviction that it is easier to admire from a distance than to practise under pressure. Looking at the outcome, over 200 million users and a project that governments mandate as their official document standard, it is clear that the decision was right. But it was not obviously right at the time, and the people who made it deserve genuine credit for that clarity.

This project has also changed how I think about my own future in software development. The tools I will build will run on open source foundations. The languages I will use were designed by communities who chose to share their work. The infrastructure I will deploy will depend on decades of contributed effort from developers who asked for nothing in return except that the tradition of sharing continue. Understanding where those tools came from — and the principles that made them possible — is not background knowledge. It is the foundation of professional responsibility in this field.

End of Report